

960121

THE “GO-LIVE” AUDIT

Session 10



OBJECTIVE

- For the final session, we move our focus from **designing** (UML) to **auditing** (The Pre-launch Checklist). In the industry, this is often called a "Go/No-Go" meeting.
- We will interrogate our own architecture to find vulnerabilities.

THE "GO-LIVE" AUDIT

This session focuses on the final polish and security measures required for a public-facing application:

- **Environment Security:** Ensuring that secrets (API keys, DB paths, JWT salts) are stored in `.env` files and never committed to GitHub.
- **Error Handling (UX vs. Dev):** Ensuring the user sees a friendly "Oops, something went wrong" while the developer gets a detailed log.
- **Dependency Management:** Cleaning up `package.json` to ensure the project is "portable" and can be installed by anyone, anywhere.

1. ENVIRONMENT SANITIZATION (THE ".ENV" LOGIC)

The most common mistake in early full-stack development is "Hard-Coding" sensitive data.

- **The Logic:** If your `JWT_SECRET` or `DB_PATH` is inside your `server.js`, it is permanently etched into your GitHub history. Even if you delete it later, it can be found in previous commits.
- **The Audit Task:** Students must ensure that their `app.js` or `server.js` uses `process.env.VARIABLE_NAME`.
- **Critical Thinking:** *"If we change our database from SQLite to a Cloud Database like PostgreSQL, do we want to change our code, or just one line in our config file?"*

2. THE ERROR HANDLING "TWO-FACE" PATTERN

In development, we want to see every detail of an error (the "Stack Trace"). In production, a stack trace is a security vulnerability because it tells hackers exactly how our code is structured.

- **The "Go-Live" Requirement:** * **Internal:** Logs the full error to a file or console for the developer.
 - **External:** Sends a generic 500 Internal Server Error to the user.
- **AI Audit Prompt:** *"Review my Express error-handling middleware. Ensure it hides stack traces from the client but logs them for the server. Help me implement a 'Production Mode' check."*

3. DEPENDENCY & PORTABILITY AUDIT

A professional project must be "Portable."

- **The Checklist:**

- Is every library used (`bcrypt`, `dotenv`, `sqlite3`) listed in `package.json`?
- Does the project run simply by typing `npm install` and `npm start`?
- Are there "absolute paths" (e.g., `C:/Users/Student/Project/...`) that will break on a different computer?

- **The "Clean Slate" Test:** Have students clone their own repo into a temporary folder and try to run it. This is the ultimate audit.

4. "SECURITY GATEKEEPER" FINAL PASS

Since students implemented a **Controller-Route-Service** pattern, the audit should look at the "Gaps" between these layers.

Audit Area	Critical Thinking Question	The "Go-Live" Fix
Input	Can a user send a 1GB string as a "Username"?	Implement <code>express-json</code> <code>limit: { limit: '10kb' }.</code>
Storage	Is the <code>.db</code> file being pushed to GitHub?	Add <code>*.db</code> to <code>.gitignore</code> .
Session	Does the JWT token ever expire?	Set an <code>expiresIn: '24h'</code> duration.

REFERENCE



CLASS ACTIVITY:

THE AI-POWERED PRE-LAUNCH AUDIT

1. The Scenario:

- **The Challenge:** Your project is "done," but is it safe? If you deploy this to the cloud right now, could a hacker steal your database? Could a user crash the site by entering a weird URL?
- **Goal:** We will create a **Safety Checklist** and use AI to audit our code against it.

THE AI-POWERED PRE-LAUNCH AUDIT

2. (15 min.) **The Manual Checklist :**

- Write down 5 things that could go "wrong" when a real user starts using their app.
- **Logic Check:** They should think about:
 - What if I forget to hide my secret key?
 - What if the database file is missing on the server?
 - What if 100 people click 'Buy' at the same time?

THE AI-POWERED PRE-LAUNCH AUDIT

3. (5 min.) **Peer Evaluation:**

- Students swap checklist.
- **Critical Thinking Question:**
 - Did your friend think about **Input Sanitization**?
 - Did your friend think about what happens if the server loses its internet connection?

THE AI-POWERED PRE-LAUNCH AUDIT

3. (5 min.) **Peer Evaluation:**

- Check your friend's code.
- **Critical Thinking Question:**
 - Is their JWT secret hard-coded in `server.js`?
 - Is their `store.db` file listed in their `.gitignore`?

THE AI-POWERED PRE-LAUNCH AUDIT

4.1. (5 min.) GenAI Prompt Engineering :

Prompt: (The "Reviewer")

“I am finishing a full-stack e-commerce project with Express and SQLite. Review my current project structure [List folders/files]. How should I set up a .env file and use the dotenv library to keep my secrets safe? Also, provide a README.md template that explains my architecture to a potential employer.”

THE AI-POWERED PRE-LAUNCH AUDIT

5.1. (10 min.) **Integration:**

- Students install the `dotenv` package.
- They create/check a `.gitignore` file to ensure the database and environment variables are excluded from version control.

THE AI-POWERED PRE-LAUNCH AUDIT

6.1. (10 min.) **Debugging & Testing:**

The “Clean Environment” Test

- **The Test 1:** Delete your local `node_modules` and try to reinstall and run the app.
- **Observation:** Does the app break because a file is missing? If yes, you must fix the dependency logic. This ensures the project is "portable."
- **The Test 2:** Clone your own repo into a temporary folder and try to run it.

THE AI-POWERED PRE-LAUNCH AUDIT

4.2. (5 min.) GenAI Prompt Engineering :

Prompt: (The "Senior Auditor")

*“I am a student developer preparing to deploy a full-stack e-commerce app (Node.js, Express, SQLite). Act as a **Senior DevOps Engineer**. Provide a 10-point 'Pre-Deployment Checklist' focusing on Security, Performance, and Error Handling. Include specific code-level checks I should perform in my Controllers and Services.”*

THE AI-POWERED PRE-LAUNCH AUDIT

5.2. (20 min.) **Integration:**

- Take that AI checklist and **audit your own code** line-by-line.
- **Task:** Find one "Hard-coded" value (like `localhost:3000`) and replace it with a variable from an `.env` file.

THE AI-POWERED PRE-LAUNCH AUDIT

6.2. (20 min.) Debugging & Testing:

The "Zero-Config" Test

- **The Test:** Try to run your app **without** your `.env` file.
- **Observation:** Does the app crash gracefully with a helpful message like "Error: DATABASE_URL is missing"? If not, they must fix the "Startup Logic."

ARCHITECTING THE API

4.3. (5 min.) GenAI Prompt Engineering:

The "AI Red Team" Audit

Prompt:

“I am the Lead Architect for an e-commerce site. Here is my current Express Controller and Route logic for the Checkout. Act as a Security Auditor. Find 3 ways an attacker could exploit this code (e.g., SQL injection, Parameter Tampering, or Missing Validation) and suggest the fix.”

[drag and drop controller, route, and services files used for checkout]

ARCHITECTING THE API

4.3. (5 min.) GenAI Prompt Engineering:

- **The Analysis:** Students must read the AI's critique and explain to their firends *why* those vulnerabilities existed.
- **The Fix:** Students apply the fixes and perform one final **Manual Test** of the checkout flow.

ARCHITECTING THE API

7. (5 min.) Version Control:

- Terminal Command:

- `git add`

- `git commit -m "chore: final security audit and environment configuration."`

- `git push.`

- VS Code:

- GitHub Desktop:

KEY TAKEAWAY

Deployment is a Responsibility:

- **Data Privacy:** Students must realize that once they deploy, they are responsible for user data.
- **The README as a Contract:** Their GitHub README.md should now be updated to include "Setup Instructions." If a recruiter can't run the project, the project effectively doesn't exist.
- **Professionalism:** Moving from "it works on my machine" to "it works everywhere."

KEY TAKEAWAY

- Software Architecture isn't just about the code you write; it's about the **environment** you build for that code to live in. A professional developer builds systems that are secure, portable, and documented.
- An architect's job doesn't end when the code is written; it ends when the system is **safe, stable, and ready for the user**. Use AI to find your blind spots before your users do.
- A 'Go-Live' Audit is about **Defensive Programming**. We assume the environment is hostile, the users make mistakes, and the server might fail. Our job is to ensure the system handles all three with grace.